Validation-Based (Optimistic) Concurrency Control Protocol

Concept

- Used when **most transactions are read-only** and **conflicts are rare**.
- Instead of locking data items, transactions **run freely** and are **checked for conflicts only at the end**.
- This reduces overhead and improves performance for systems with **low contention**.

That's why it's called **Optimistic Concurrency Control** — it assumes everything will go well ("optimistically").

Phases of Execution

Each transaction **Ti** passes through **three phases**:

1. Read Phase

- The transaction **reads data items** from the database.
- **All updates are done on local copies** (temporary variables).
- The database itself is **not modified yet**.

2. Validation Phase

- Before writing results to the database, the system **checks for conflicts**.
- If no conflict → the transaction **passes validation**.
- If conflict is found → the transaction **fails validation** and is **aborted (rolled back)**.

3. Write Phase

- If validation is successful → **copy local changes to the actual database**.
- If the transaction is read-only → this phase is **skipped**.

Timestamps Used

Each transaction **Ti** has three timestamps:

Timestamp	Meaning
`Start(Ti)`	Time when transaction starts.
`Validation(Ti)`	Time when read phase ends (used as timestamp TS(Ti)).
`Finish(Ti)`	Time when write phase ends.

The **serial order** is determined by **Validation(Ti)** — not Start(Ti).

Validation Test Conditions

For each transaction **Ti**, and for every **earlier transaction Tk** (with `TS(Tk) < TS(Ti)`), one of these must be **true**:

1. **Condition 1:**
   `Finish(Tk) < Start(Ti)`
   → Tk finished before Ti started → no overlap → serial order preserved.

2. **Condition 2:**

   - Tk's **write set** does **not intersect** with Ti's **read set**, and

   - `Start(Ti) < Finish(Tk) < Validation(Ti)`
     → Tk's writes don't affect Ti's reads → still serializable.

If neither condition holds → validation fails → transaction **Ti is rolled back**.

Example:

T_{25}	T_{26}
read(B)	
	read(B)
	$B := B - 50$
	read(A)
	$A := A + 50$
read(A)	
< validate>	
display($A + B$)	
	< validate>
	write(B)
	write(A)

Two transactions are running **at the same time**:

- **T25** → Only *reads* A and B (read-only transaction).
- **T26** → *Transfers ₹50 from B to A* (updates both A and B).

T25 (Read-Only Transaction)

```
read(B)

read(A)

< validate >

display(A + B)
```

- T25 reads both account balances (A and B).
- Then it validates and displays the total.
- It **does not change anything** in the database — only reads.

T26 (Update Transaction)

```
read(B)
B := B - 50
read(A)
A := A + 50
< validate >
write(B)
write(A)
```

- T26 reads both accounts.
- Subtracts ₹50 from B and adds ₹50 to A (like a money transfer).
- Then validates, and if it passes → writes (updates) the actual database.

What happens in order

Let's assume both are running at the same time.

1. **T25** starts and reads **A and B**.
   (Let's say A = 1000, B = 500)
2. **T26** also starts:
   - Reads **B = 500**, updates local copy → B = 450
   - Reads **A = 1000**, updates local copy → A = 1050
3. **T25 validates first** and finishes:
   - It only read old values (A=1000, B=500)
   - Displays total = **1500**
4. **T26 validates next:**
   - Validation test checks that T25 (older) and T26 (newer) don't conflict.

- T25 is read-only → so no problem.
- T26 passes validation.

5. **T26 writes updates:**
   - B = 450
   - A = 1050

Result

Account	Final Value
A	1050
B	450
Total	1500

Validation Test for T26

For every earlier transaction (**Tk**) with `TS(Tk) < TS(T26)` (here, Tk = T25):

We check these **two conditions** — at least one must be true:

1. **Finish(Tk) < Start(Ti)**
   → Did T25 finish before T26 started?
   No — they overlapped.

2. **Start(Ti) < Finish(Tk) < Validation(Ti)**
   → T25 didn't write anything (it only read),
   → So there's **no conflict**

Validation succeeds for T26

Timestamps Summary Table

Transaction	Start	Validation	Finish	Remarks
T25	1	2	3	Read-only, commits early
T26	2	4	5	Overlaps with T25, passes validation